

# Chunking and Embedding Strategies for RAG

## A technical evaluation, with measurements from the Marginalia project

6 September 2026

### Contents

1. **Summary** — recommendations and the three findings that matter most
2. **Method** — corpus, metrics, harness, controls
3. **Part One: Chunking Strategies** — fixed, semantic, hierarchical; measured results; the placement-not-size finding
4. **Part Two: Embedding Providers** — what differentiates them, pricing, cost on a real corpus
5. **Part Three: Vector Stores** — free tiers and what they cost at scale
6. **Part Four: Answer Generation** — pricing, cost per question, fine-tuning economics
7. **Part Five: Total Cost of Ownership** — three configurations compared
8. **Part Six: Recommendation**
9. **Appendix A** — reproducing these measurements
10. **Appendix B** — sources

### 1. Summary

This report compares three chunking strategies and six embedding providers for a retrieval-augmented generation (RAG) system. Unlike most such comparisons, the chunking section is not built on published benchmarks: every figure comes from A/B runs on this project's own corpus, using its own 26-question evaluation set.

### Recommendations

| Layer               | Recommendation                                                                                                              | Basis                                                                                              |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Chunking</b>     | <b>Semantic</b> , if a ~450-second one-time ingest cost is acceptable. Otherwise <b>fixed</b> .                             | Measured: +0.093 MRR over fixed at production settings                                             |
| <b>Embeddings</b>   | <b>Local bge-small</b> while the corpus is small; <b>Gemini</b> or <b>Voyage 4</b> when quality matters more than zero cost | Embedding cost is negligible at this scale; the decision is quality and operational, not financial |
| <b>Vector store</b> | <b>Pinecone Starter</b> (free) or <b>Chroma Cloud</b> (~\$0.25/mo) at current size                                          | Both are effectively free below 2 GB                                                               |
| <b>Generation</b>   | <b>Groq gpt-oss-20b</b>                                                                                                     | 22× cheaper than Gemini 3.5 Flash, 33× cheaper than a tuned Gemini endpoint                        |

## The three findings that matter most

**Hierarchical (parent-child) chunking lost, and the way it lost was informative.** It ranked worse than both alternatives while requiring 75% more vectors. But because its children are effectively “fixed chunking at 150 words”, it accidentally supplied a control that had never been run — and that control showed **semantic chunking’s advantage comes from where it places boundaries, not from producing smaller chunks**. A cheap configuration change (`CHUNK_SIZE_WORDS=175`) would therefore *not* have captured the benefit.

**Embedding cost is not a real decision variable at this scale.** A complete re-embed of the 500-page test corpus costs approximately **\$0.04** with Gemini. Even semantic chunking, which embeds every sentence and multiplies token consumption roughly fifteen-fold, costs about **\$0.64** per full ingest. Provider choice should be made on quality, dimensions and operational fit — not price.

**A fine-tuned Gemini endpoint costs 33× more per question than Groq.** As of Gemini 3, Google bills tuned model endpoints at **1.5× the base model rate**. Combined with Gemini 3.5 Flash already being 22× Groq’s price, a tuned endpoint runs about **\$0.0142 per question** against Groq’s **\$0.0004**. Fine-tuning is defensible for voice and answer shape; it is not defensible as a cost optimisation.

---

## 2. Method

**Corpus.** Two technical textbooks, first 250 extracted pages each — roughly 500 pages and 219,000 words.

**Evaluation set.** 26 questions with verified source document and page numbers, plus 3 deliberately unanswerable questions for measuring refusal behaviour. Every question carries an evidence list — literal strings that must appear on the cited page — verified by re-extracting the PDFs.

### Metrics.

- **hit-rate@k** — was a chunk covering the expected page retrieved at all?
- **MRR** (Mean Reciprocal Rank) — how highly was it ranked? MRR is the stricter and more informative metric, because it only counts chunks the system actually *ranked*, ignoring those that arrived through neighbour or parent expansion.
- **ctx words** — median words of context delivered to the model per question. Included because a strategy that scores higher while sending twice the text has not necessarily won: some of that gain is available to any strategy simply by raising `top_k`.

**Harness.** Chunking is compared offline by exact brute-force cosine search over vectors held in memory. Nothing is written to any index. Two reasons: production vector stores are approximate, and their run-to-run recall variance is roughly the size of the effect being measured; and a comparison script that writes vectors is a script that can leave two strategies’ vectors mixed in one collection — which produces confident nonsense with no error anywhere.

**Controls.** Same embedding model (BAAI/bge-small-en-v1.5, local), same `top_k`, same re-ranker, same questions across all arms. One variable per run.

**Prices** were read from vendors’ official pricing pages on **6 September 2026** and are cited individually. Where a vendor does not publish a rate, this report says so rather than estimating.

---

## 3. Part One — Chunking Strategies

### 3.1 The problem

An embedding model maps a passage to a single fixed-length vector. Too large a passage and its vector becomes an average of several topics, matching every query weakly and none strongly. Too small and it loses the context that makes it meaningful.

The decision is harder than it looks because **one chunk size serves two conflicting jobs**:

- what gets **embedded and ranked** wants to be *small* — a tight vector, a narrow page range
- what the model **reads to answer** wants to be *large* — a complete argument, not half of one

Every strategy below is a different answer to that tension.

### 3.2 Fixed-size chunking

**Mechanism.** Pack whole paragraphs into chunks up to a word limit, carrying the trailing ~50 words into the next chunk as overlap. Boundaries fall wherever the word count runs out.

| Pros                                        | Cons                                                                       |
|---------------------------------------------|----------------------------------------------------------------------------|
| Fast — sub-second for a 500-page corpus     | Boundaries ignore meaning entirely                                         |
| Completely predictable chunk count and cost | A boundary can split an argument in half, leaving neither part retrievable |
| No embedding calls during chunking          | Overlap is a patch over that, and duplicates text                          |
| Trivial to reason about and debug           | One size must serve both jobs above                                        |

**Measured on this corpus:** 1,030 chunks, median 254.5 words.

**An observation worth recording:** 169 of those 1,030 chunks (16%) exceeded the embedding model's 512-token window and were silently re-split at embedding time. `CHUNK_SIZE_WORDS=300` is therefore *not* the boundary it appears to be on dense technical text. This is invisible without checking the ingest log.

### 3.3 Semantic chunking

**Mechanism.** Split the document into sentences. Embed each sentence together with its immediate neighbours. Measure cosine distance between consecutive windows. Place a boundary wherever that distance exceeds the 95th percentile *of that document's own distances*.

A percentile rather than a fixed threshold, because absolute distances shift with the document and the embedding model: a constant that splits sensibly in one book cuts every other sentence in the next.

| Pros                                                | Cons                                                       |
|-----------------------------------------------------|------------------------------------------------------------|
| Boundaries fall where the topic actually changes    | ~15× the embedding calls at ingest — one per sentence      |
| <b>Best measured quality</b> on this corpus         | Slowest by a wide margin: ~455 s vs ~0.2 s                 |
| No overlap needed — the boundary is already correct | Quality depends on the embedding model doing the measuring |
| Ingest cost is one-time; nothing extra per query    | Harder to predict chunk count in advance                   |

**Measured on this corpus:** 1,235 chunks, median 174 words, ~455 s to chunk.

### 3.4 Hierarchical (parent-child) chunking

**Mechanism.** Pack text into small **children** (~150 words) and group those children into larger **parents** (~600 words). Embed and rank only the children; when one is retrieved, expand it to its whole parent before the model reads it. Small unit for precision, large unit for context — the explicit answer to the two-jobs problem.

In this implementation parents are never stored. A parent is simply a run of consecutive children sharing a `parent_index`, reconstructed at query time by fetching the hit's siblings.

| Pros                                                | Cons                                                 |
|-----------------------------------------------------|------------------------------------------------------|
| Directly addresses the size conflict                | <b>Worst MRR of the three</b> on this corpus         |
| Chunking cost identical to fixed — pure string work | 75% more vectors to store and embed                  |
| Children never exceed the token window              | Delivers ~2× the context, costing tokens and latency |
| No overlap needed — the parent supplies context     | Boundaries are still word-count based, not semantic  |

**Measured on this corpus:** 1,799 chunks in 409 parents, median 133 words.

### 3.5 Measured results

**Run 1 — production settings** (top\_k=4, re-ranking on, neighbour expansion 1):

| Strategy        | Chunks | Median words | Context words | hit@4        | MRR          |
|-----------------|--------|--------------|---------------|--------------|--------------|
| fixed           | 1,030  | 254.5        | 2,353         | 0.923        | 0.788        |
| <b>semantic</b> | 1,235  | 174          | 1,807         | <b>0.962</b> | <b>0.881</b> |
| hierarchical    | 1,799  | 133          | 1,680         | 0.923        | 0.724        |

**Run 2 — safety nets removed** (top\_k=1, no re-ranking, no expansion):

| Strategy        | Context words | hit@1        | MRR          | Hits by ranking | Hits by expansion |
|-----------------|---------------|--------------|--------------|-----------------|-------------------|
| fixed           | 278           | 0.692        | 0.692        | 18              | 0                 |
| <b>semantic</b> | 232           | <b>0.846</b> | <b>0.846</b> | 22              | 0                 |
| hierarchical    | 550           | 0.846        | 0.654        | 17              | 5                 |

All three arms chunk approximately 219,000 words, confirming no arm gains by silently dropping text.

**Run 2's apparent tie is not a tie.** Semantic and hierarchical both show 0.846 hit-rate, but semantic found 22 of those answers *by ranking them*, while hierarchical ranked only 17 and obtained 5 through parent expansion — using 2.4× the context. MRR, which ignores expansion, separates them cleanly.

### 3.6 The finding: placement, not size

Before these runs, an open question remained: is semantic chunking's rank-1 advantage caused by **where it puts boundaries**, or merely by **producing smaller chunks**? The distinction carries very different prices — one line of configuration versus 455 seconds of sentence embedding per ingest.

The hierarchical run answered it as a side effect. A hierarchical child is essentially “fixed chunking at 150 words with no overlap” — median 133 words, *smaller* than semantic's 174. If small chunks were the cause, hierarchical should have approached semantic's 0.846 MRR.

It scored **0.654** — adjacent to fixed's 0.692, nowhere near semantic's 0.846.

**Conclusion: boundary placement earns the advantage.** Setting `CHUNK_SIZE_WORDS=175` would not have captured it. Semantic chunking's cost buys something a configuration change cannot.

### 3.7 Verdict on chunking

|                 | Quality      | Ingest cost     | Storage  | Recommendation                            |
|-----------------|--------------|-----------------|----------|-------------------------------------------|
| fixed           | Baseline     | Lowest          | Lowest   | Safe default; fine for most corpora       |
| <b>semantic</b> | <b>Best</b>  | High (one-time) | Moderate | <b>Use when retrieval quality matters</b> |
| hierarchical    | Worst on MRR | Low             | Highest  | Not recommended on this evidence          |

Semantic chunking's cost is paid **once, at ingest**, and nothing extra per query. For a corpus that changes rarely — a product knowledge base, a document library — 455 seconds is a negligible price for +0.093 MRR on every question thereafter.

Hierarchical is not recommended here, but the concept is not discredited: its children used word-count boundaries. **Semantic children inside structural parents** is the untested combination that would isolate whether the parent-child idea has value once the boundary problem is solved separately.

### 3.8 Caveats

- **26 questions is a small evaluation set.** The smallest difference it can express is one question, or 0.038. Differences smaller than roughly 0.077 should be treated as noise.
  - **The fixed baseline moved between test sessions** — 1.000/0.942 in an earlier run against 0.923/0.788 here. Something changed in the environment (question set, embedding model, or re-ranker). Numbers should not be compared across those two sessions.
  - **One question fails in all three strategies in both runs.** Six independent configurations agreeing points to an incorrect page number in the evaluation set rather than a retrieval failure.
  - **Results are corpus-dependent.** This corpus is well-structured technical prose with intact paragraphs. Published comparisons generally find semantic chunking clearly ahead only where formatting carries no signal — transcripts, chat logs, OCR without paragraph breaks. That semantic wins measurably *here* is the more interesting result.
-

## 4. Part Two — Embedding Providers

### 4.1 What actually differentiates them

**Dimensions** determine storage and search cost. A 3072-dimension vector costs four times a 768-dimension one in both index size and comparison work. Several current models are *Matryoshka* — trained so that any prefix of the vector remains usable, making dimension a tunable dial rather than a fixed property.

**Query/passage asymmetry.** Every serious retrieval model embeds a *question* differently from a *passage*. Getting this wrong returns perfectly valid vectors and ranks them measurably worse, with nothing in any log. Each provider spells it differently — `task_type`, `input_type`, `task` — and a local BGE model uses an instruction prefix on the query only.

**Context window** caps chunk size. BGE's 512 tokens is far shorter than every hosted alternative, and it is what made 16% of 300-word chunks overflow in the fixed-chunking run.

**Normalisation.** Gemini's `gemini-embedding-001` returns **unnormalised** vectors at any dimension other than 3072. Since cosine similarity on unnormalised vectors ranks partly by magnitude — which for text tracks length — omitting the L2 normalisation step causes long chunks to outrank relevant ones, silently.

### 4.2 Pricing

Observed 6 September 2026 from official pricing pages.

| Provider / model                   | \$/1M tokens                  | Free tier          | Dimensions                           | Context |
|------------------------------------|-------------------------------|--------------------|--------------------------------------|---------|
| <b>Local</b> bge-small-en-v1.5     | <b>\$0</b>                    | n/a (Apache-2.0)   | 384 fixed                            | 512     |
| Local bge-base-en-v1.5             | \$0                           | n/a                | 768 fixed                            | 512     |
| OpenAI embedding-3-small           | <b>\$0.02</b>                 | None               | 1536 (MRL)                           | 8,192   |
| <b>Voyage</b> voyage-4-lite        | <b>\$0.02</b>                 | <b>200M tokens</b> | 1024, 256–2048 (MRL)                 | 32,000  |
| <b>Voyage</b> voyage-4             | <b>\$0.06</b>                 | <b>200M tokens</b> | 1024, 256–2048 (MRL)                 | 32,000  |
| <b>Voyage</b> voyage-4-large       | \$0.12                        | 200M tokens        | 1024, 256–2048 (MRL)                 | 32,000  |
| OpenAI embedding-3-large           | \$0.13                        | None               | 3072 (MRL)                           | 8,192   |
| <b>Google</b> gemini-embedding-001 | <b>\$0.15</b> (\$0.075 batch) | Yes, rate-limited  | <b>128–3072</b> , default 3072 (MRL) | 2,048   |
| Google Vertex, same model          | \$0.15(\$0.12 batch)          | GCP credits only   | same                                 | 2,048   |

| Provider / model                    | \$/1M tokens         | Free tier              | Dimensions           | Context |
|-------------------------------------|----------------------|------------------------|----------------------|---------|
| <b>Pinecone</b> llama-text-embed-v2 | <b>\$0.16</b>        | 5M tokens/mo           | 1024, 384–2048 (MRL) | 2,048   |
| Cohere embed-v4.0                   | <b>not published</b> | Trial (non-commercial) | 1536, 256–1536 (MRL) | 128,000 |
| Jina embeddings-v5                  | <b>not published</b> | 10M tokens             | 1024 (MRL)           | 32,768  |

*MRL = Matryoshka: any prefix of the vector is usable, so the dimension is a tunable dial.*

Cohere and Jina no longer publish per-token embedding rates on their public pricing pages; both defer to sales or marketplace listings. That is reported as observed rather than estimated.

### 4.3 What this costs on a real corpus

The 500-page test corpus is approximately 219,000 words  $\approx$  285,000 tokens.

| Strategy     | Tokens embedded per full ingest   | Gemini (\$0.15/1M) | Voyage 4 (\$0.06/1M) | Local |
|--------------|-----------------------------------|--------------------|----------------------|-------|
| fixed        | $\sim$ 285,000                    | <b>\$0.04</b>      | \$0.02               | \$0   |
| hierarchical | $\sim$ 285,000                    | <b>\$0.04</b>      | \$0.02               | \$0   |
| semantic     | $\sim$ 4,300,000 (sentence-level) | <b>\$0.64</b>      | \$0.26               | \$0   |

Query-side cost is negligible: a question is roughly 20 tokens, so 10,000 questions consume about 200,000 tokens — **\$0.03** with Gemini.

**The conclusion is that embedding price is not a decision variable at this scale.** The difference between the cheapest and most expensive credible option, across a full re-ingest plus 10,000 questions, is under one dollar.

### 4.4 Provider assessment

**Local (bge-small-en-v1.5)** — Free, private, no network dependency, no rate limits. Costs CPU and a  $\sim$ 130 MB model load at startup. Its 512-token window is the tightest here and constrains chunk size. Only 384 dimensions, which limits expressiveness but makes the index small. *Best for development, privacy-sensitive deployments, and any corpus small enough that CPU time is not a constraint.*

**Google Gemini** — Flexible dimensions (128–3072), a genuine free tier, and quality among the strongest available. Three operational traps: non-3072 dimensions return unnormalised and **must** be normalised by the caller; the 2,048-token window is mid-range; and free-tier per-minute token limits are low enough that a large batch can exceed the budget in a single request. *Best when dimension flexibility matters and a free tier is useful.*

**Voyage AI** — The strongest price-to-capability ratio in the table: voyage-4 at \$0.06/1M with a 32,000-token window and a **200 million token** free allowance — enough to embed this corpus roughly 700 times over. *Best value if you are paying.*

**OpenAI** — `text-embedding-3-small` at \$0.02/1M is the cheapest paid option with an 8,192-token window. No free tier, and no query/passage asymmetry, which is a genuine (if modest) quality disadvantage for retrieval. *Best when you are already an OpenAI customer.*

**Pinecone** — The most expensive at \$0.16/1M, and its 2,048-token window matches Gemini's. Its argument is not price but **co-location**: embeddings, vector storage and re-ranking from one vendor, one key, one bill. *Best when operational simplicity outweighs unit cost.*

**Cohere** — 128,000-token context is by far the largest here, which matters if you intend to embed whole documents rather than chunks. Unpublished pricing and non-commercial trial keys make it hard to evaluate or adopt without a sales conversation.

#### 4.5 Verdict on embeddings

| Situation                                | Choose                                                    |
|------------------------------------------|-----------------------------------------------------------|
| Development, privacy, or zero budget     | <b>Local bge-small</b>                                    |
| Best value when paying                   | <b>Voyage voyage-4</b> — \$0.06/1M, 200M free, 32k window |
| Flexible dimensions, free tier           | <b>Gemini gemini-embedding-001</b>                        |
| Cheapest paid, already on OpenAI         | <code>text-embedding-3-small</code>                       |
| One vendor for the whole retrieval stack | Pinecone                                                  |

**Changing embedding provider invalidates every stored vector.** Vectors from two models are not comparable, and mixing them produces no error — retrieval simply degrades. Any switch requires a full re-ingest into a *new* collection.

### 5. Part Three — Vector Stores

Sizing basis: 250,000 vectors at 768 dimensions ≈ 0.75 GB raw, 1–1.5 GB with index overhead.

| Store                      | Free tier                    | Paid model                                                 | Est. monthly at this size         |
|----------------------------|------------------------------|------------------------------------------------------------|-----------------------------------|
| <b>Pinecone Serverless</b> | 2 GB, 2M writes, 1M reads/mo | Builder \$20/mo; Standard \$50/mo minimum                  | <b>\$0</b> on Starter             |
| <b>Chroma Cloud</b>        | \$5 credit, \$0/mo + usage   | \$0.33/GiB storage, \$2.50/GiB write, \$0.0075/TiB queried | <b>~\$0.25/mo</b>                 |
| <b>Qdrant Cloud</b>        | 1 GB RAM node, free forever  | Hourly resource-based — <b>rates not published</b>         | Not determinable                  |
| <b>Weaviate Cloud</b>      | Sandbox: 100k objects        | From \$0.00465/1M dimensions, <b>\$45/mo floor</b>         | <b>\$45/mo</b> (floor, not usage) |
| <b>Self-hosted</b>         | n/a — open source            | Server only                                                | <b>\$12–24/mo</b> VPS             |

Pinecone and Qdrant have both moved per-unit rates behind interactive calculators, so above-floor costs cannot be computed from published figures.

**Assessment.** At this scale the decision is not financial — Pinecone Starter is free and Chroma Cloud costs pennies. Weaviate’s \$45/mo floor is 50× its actual metered usage here, making it poor value below roughly 10 million vectors. Self-hosting is competitive only when you already run servers.

## 6. Part Four — Answer Generation

### 6.1 Pricing

| Provider / model              | Input \$/1M                                     | Output \$/1M  | Free tier               |
|-------------------------------|-------------------------------------------------|---------------|-------------------------|
| <b>Groq gpt-oss-20b</b>       | <b>\$0.075</b>                                  | <b>\$0.30</b> | 30 RPM, 200k tokens/day |
| Groq gpt-oss-120b             | \$0.15                                          | \$0.60        | Yes                     |
| Groq llama-3.3-70b            | <b>no longer published</b><br>— “Contact Sales” | —             | —                       |
| Google gemini-3.5-flash-lite  | \$0.30                                          | \$2.50        | Yes                     |
| Google gemini-3.5-flash       | \$0.75*                                         | \$3.75*       | Yes                     |
| Google gemini-3.5-flash       | \$1.50                                          | \$9.00        | Yes                     |
| Google gemini-3.1-pro-preview | \$2.00                                          | \$12.00       | No                      |

\* Introductory pricing through 31 December 2026; reverts to \$1.50 / \$7.50.

### 6.2 Cost per question

Typical question at production settings: ~4,500 input tokens (system prompt plus ~2,350 words of retrieved context) and ~300 output tokens.

| Model                         | Per question     | Per 1,000 questions | vs Groq    |
|-------------------------------|------------------|---------------------|------------|
| <b>Groq gpt-oss-20b</b>       | <b>\$0.00043</b> | <b>\$0.43</b>       | —          |
| Gemini 3.5 Flash-Lite         | \$0.0021         | \$2.10              | 5×         |
| Gemini 3.5 Flash              | \$0.0095         | \$9.45              | 22×        |
| <b>Tuned Gemini 3.5 Flash</b> | <b>\$0.0142</b>  | <b>\$14.18</b>      | <b>33×</b> |

### 6.3 Fine-tuning economics

Google removed fine-tuning from the Gemini API and AI Studio in May 2025; supervised tuning now runs on **Vertex AI**, which uses OAuth service-account credentials rather than an API key, and a region-specific host.

**Training cost** on this project’s dataset — 599 examples, ~316,000 tokens per epoch:

| Epochs | Training tokens | Cost (Gemini 3.5 Flash SFT, \$10/1M) |
|--------|-----------------|--------------------------------------|
| 3      | 0.95M           | <b>\$9.47</b>                        |
| 5      | 1.58M           | <b>\$15.78</b>                       |

Training is inexpensive and one-time. **Serving is where the cost lives.** From Vertex’s pricing page:

“For model inference starting from Gemini 3, tuned model endpoint prediction price will be 1.5 times of the base model. Old Gemini models prediction price stays the same as the base model.”

So a tuned Gemini 3.5 Flash endpoint bills at \$2.25 in / \$13.50 out per 1M tokens — and at 1,000 questions per month costs **\$14.18 against Groq’s \$0.43**. No separate hosting or idle endpoint charge is published; the cost is entirely per-token.

#### 6.4 When fine-tuning is worth it — and when it is not

**A caution that applies directly here.** The training data for this project consists of *question → answer* pairs with no retrieved context in the prompt. That teaches the model to answer **from its weights**. A RAG system’s entire contract is the opposite: answer only from the retrieved passages, and decline when they do not cover the question.

Deploying a weights-trained model inside a RAG pipeline produces three specific failures:

1. It answers confidently when retrieval returns nothing, defeating the refusal path.
2. Citations become decorative — it cites a retrieved page while answering from memory.
3. **Facts go stale in the weights.** A price corrected in a source PDF updates on the next ingest; the tuned model keeps repeating the trained value until retrained.

**Fine-tuning is worth it for voice and answer shape** — house style, formatting, tone, consistent structure. To get that without sacrificing grounding, the training data must include the CONTEXT block, so each example teaches “*given these passages, answer in this style*” rather than “*answer this question from memory.*”

## 7. Part Five — Total Cost of Ownership

Three configurations, at 1,000 questions per month over a corpus of ~250,000 vectors.

|                         | A: Zero-cost     | B: Balanced      | C: Fine-tuned          |
|-------------------------|------------------|------------------|------------------------|
| Chunking                | semantic         | semantic         | semantic               |
| Embeddings              | local bge-small  | Gemini           | Gemini                 |
| Vector store            | local Chroma     | Pinecone Starter | Pinecone Starter       |
| Generation              | Groq gpt-oss-20b | Groq gpt-oss-20b | Tuned Gemini 3.5 Flash |
| <b>Embedding /mo</b>    | \$0              | ~\$0.04          | ~\$0.04                |
| <b>Vector store /mo</b> | \$0              | \$0              | \$0                    |
| <b>Generation /mo</b>   | \$0.43           | \$0.43           | \$14.18                |

|                  | A: Zero-cost | B: Balanced    | C: Fine-tuned    |
|------------------|--------------|----------------|------------------|
| <b>Total /mo</b> | <b>~\$0</b>  | <b>~\$0.50</b> | <b>~\$14.20</b>  |
| One-time         | —            | —              | +\$9.47 training |

**Generation dominates.** Embeddings and vector storage together are under 10% of even the cheapest configuration's cost. Effort spent optimising embedding spend is misdirected; effort spent on model choice for generation is not.

## 8. Part Six — Recommendation

### Immediate

**Switch chunking to semantic.** It is the only change measured to improve retrieval quality on this corpus, its cost is one-time, and the +0.093 MRR applies to every question thereafter.

```
CHUNK_STRATEGY=semantic
CHROMA_COLLECTION=rag_semantic          # a new collection – required
```

Then re-ingest with `python scripts/ingest.py --force`.

**Keep Groq for generation.** At 22–33× cheaper than the Gemini alternatives with no measured quality deficit for grounded question-answering, there is no case for switching on cost, and no evidence yet for switching on quality.

**Reduce RETRIEVAL\_CANDIDATES from 30 to 15.** Re-ranking 30 candidates costs ~2.3 seconds on every question. With `top_k=4`, 15 candidates still leaves nearly 4× headroom.

### Worth testing

**Voyage voyage-4** as an embedding provider — \$0.06/1M, a 32,000-token window (which would eliminate the chunk-overflow problem entirely), and a 200M-token free allowance that covers this corpus hundreds of times over.

**Semantic children inside structural parents.** The hierarchical experiment failed because its children used word-count boundaries. Combining semantic boundaries with parent expansion is the untested variant that would show whether parent-child has value once boundary placement is handled properly.

### Not recommended

**Hierarchical chunking as measured** — worse ranking, 75% more vectors, no compensating benefit.

**A fine-tuned model as a cost or accuracy measure** — 33× the per-question cost, and training data that rewards answering from memory works directly against the grounding contract. Revisit only if the goal is explicitly *voice*, and only with training data that includes the retrieved context.

## Appendix A — Reproducing these measurements

```
# Chunking comparison – writes nothing, touches no index
EMBEDDINGS_PROVIDER=local RERANKER_PROVIDER=local \
```

```
python scripts/ab_chunking.py --max-pages 250 \  
  --strategies fixed,semantic,hierarchical --out eval/runs/run1.json
```

*# Same, with re-ranking and expansion disabled – higher resolution*

```
EMBEDDINGS_PROVIDER=local RERANKER_PROVIDER=local \  
python scripts/ab_chunking.py --max-pages 250 \  
  --strategies fixed,semantic,hierarchical \  
  --top-k 1 --no-rerank --expand 0 --out eval/runs/run2.json
```

*# End-to-end evaluation against a live index*

```
python eval/run_eval.py --out eval/runs/baseline.json
```

*# Verify the evaluation set's page numbers are correct*

```
python scripts/check_golden.py
```

**Hold everything else constant.** One variable per run. Changing chunking and the embedding model together makes the result unattributable to either.

## Appendix B — Sources

All prices observed 6 September 2026.

**Embeddings:** [Gemini API pricing](#) · [Vertex AI pricing](#) · [OpenAI pricing](#) · [Voyage AI pricing](#) · [Cohere pricing](#) · [Jina embeddings](#) · [Pinecone llama-text-embed-v2](#) · [BAAI bge-small-en-v1.5](#)

**Vector stores:** [Pinecone](#) · [Chroma Cloud](#) · [Qdrant](#) · [Weaviate](#) · [DigitalOcean droplets](#)

**Generation:** [Groq models](#) · [Groq rate limits](#) · [Vertex AI tuning and serving](#)

**Method:** [Gemini embeddings guide](#) · [Gemini fine-tuning](#)